



Преподаватель:
Коляда
Никита Владимирович

Обратная связь:
• сообщения на inStudy

ПАТТЕРН MVC И МАРШРУТИЗАЦИЯ

—

РАЗДЕЛЕНИЕ ОТВЕТСТВЕННОСТИ МЕЖДУ МОДУЛЯМИ

2026 Г.

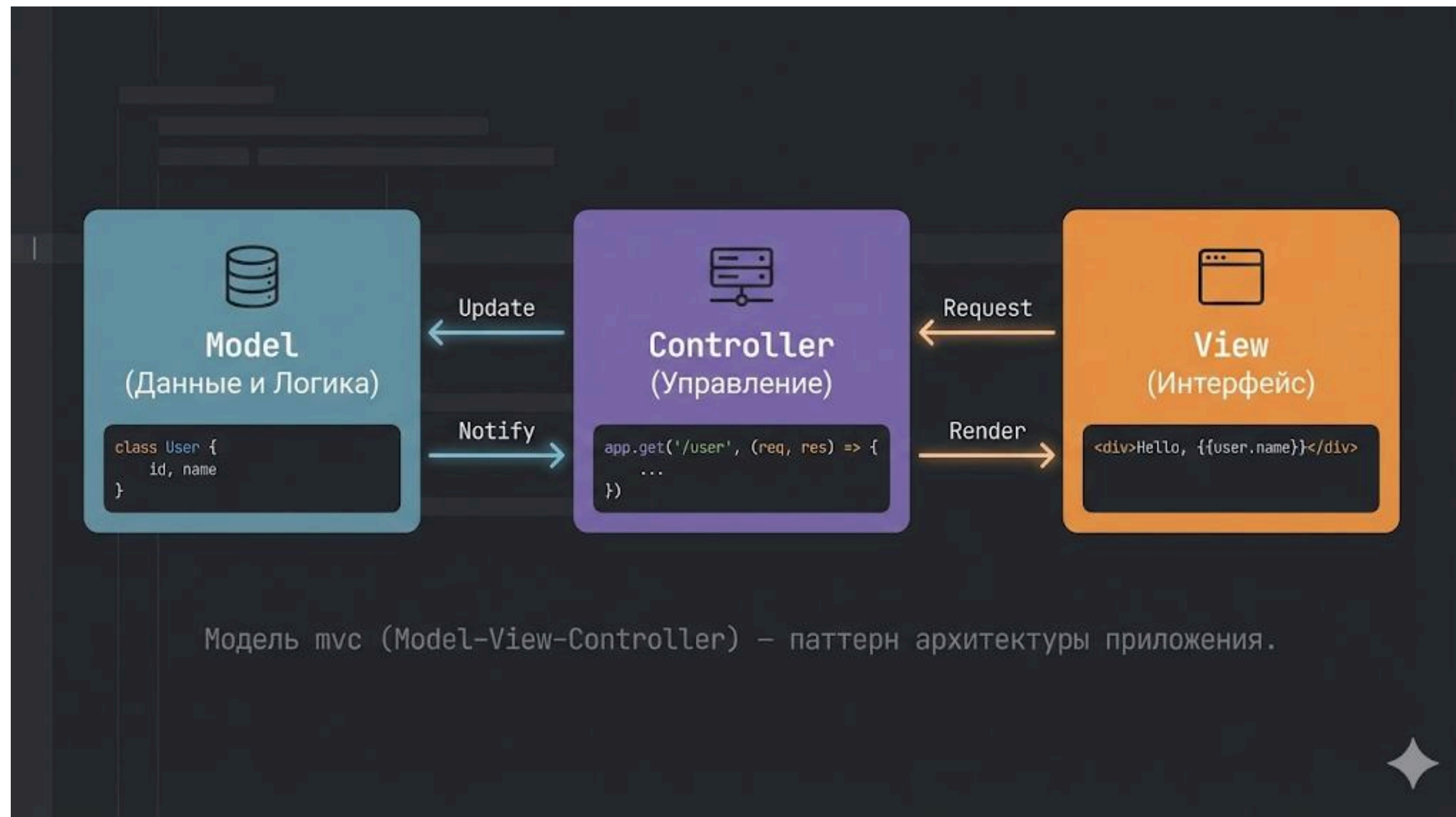


ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?

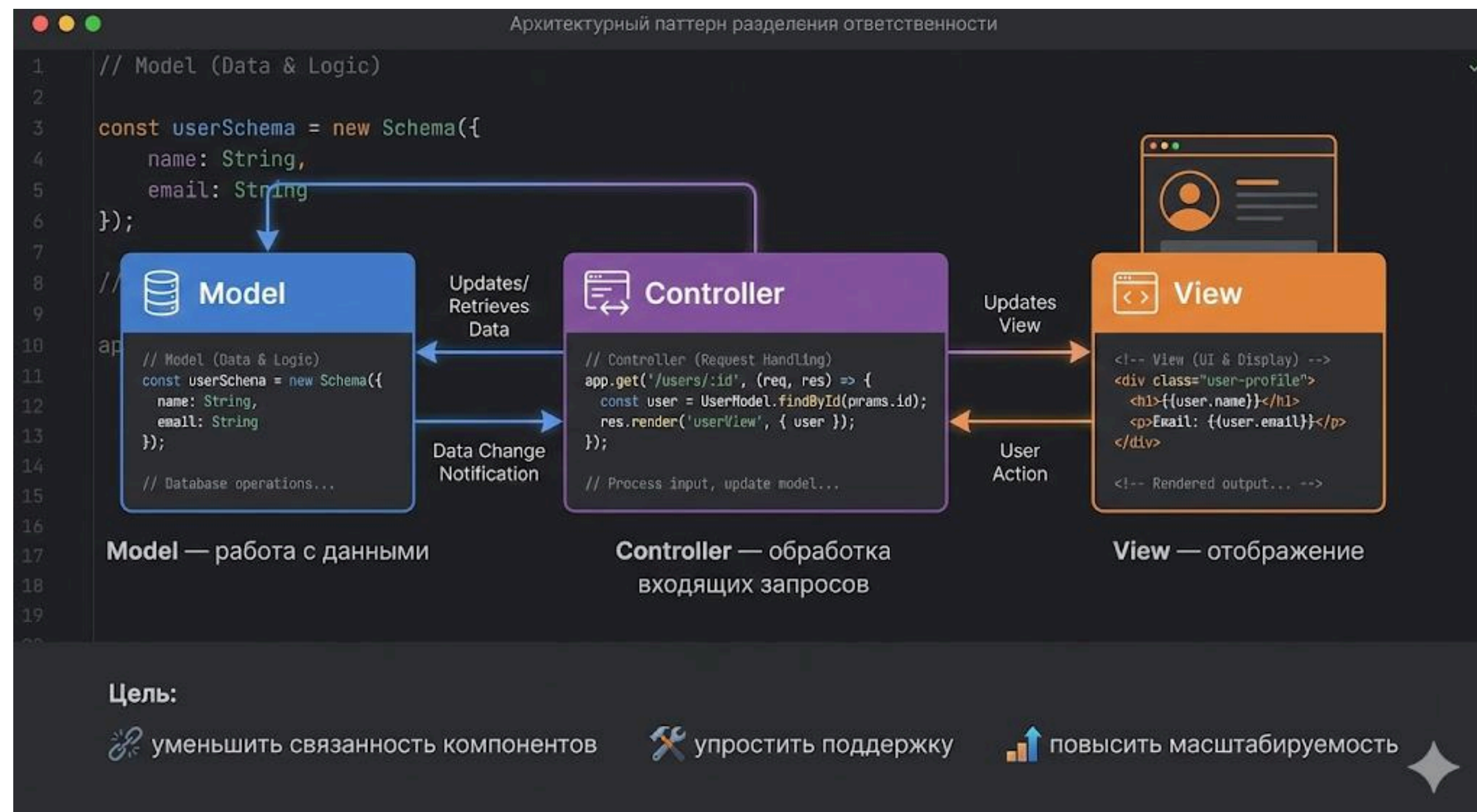
МОДЕЛЬ MVC (MODEL-VIEW-CONTROLLER)

–
ПАТТЕРН АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ



ЧТО ТАКОЕ MVC

АРХИТЕКТУРНЫЙ ПАТТЕРН РАЗДЕЛЕНИЯ ОТВЕТСТВЕННОСТИ



1. MVC (Model-View-Controller) — это способ организации кода, при котором приложение делится на три независимых слоя:

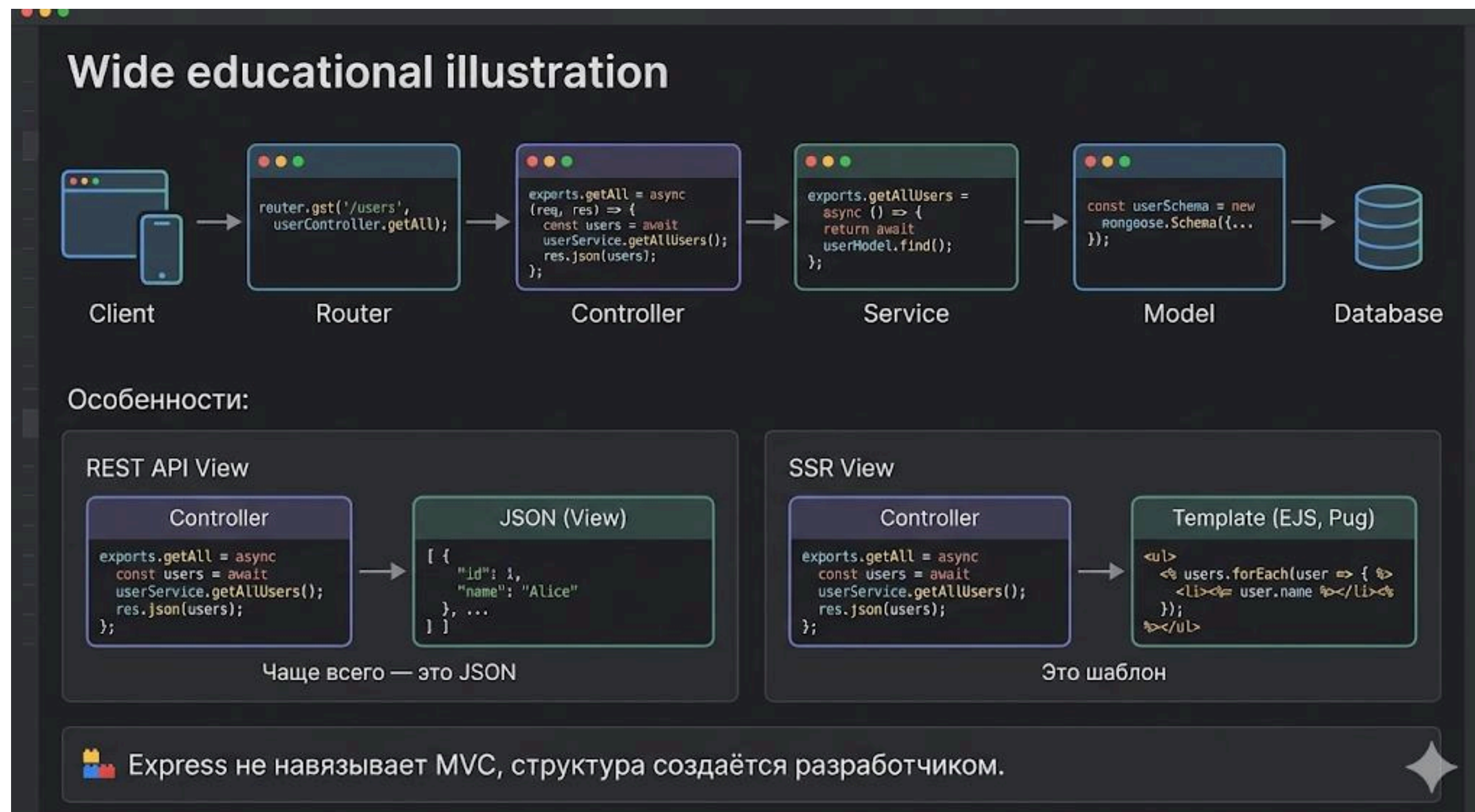
- Model — работа с данными
- View — отображение
- Controller — обработка входящих запросов

2. Цель:

- уменьшить связанность компонентов
- упростить поддержку
- повысить масштабируемость

MVC В NODE.JS

КАК ПАТТЕРН РЕАЛИЗУЕТСЯ В EXPRESS-ПРИЛОЖЕНИИ



1. Типичная схема в backend:

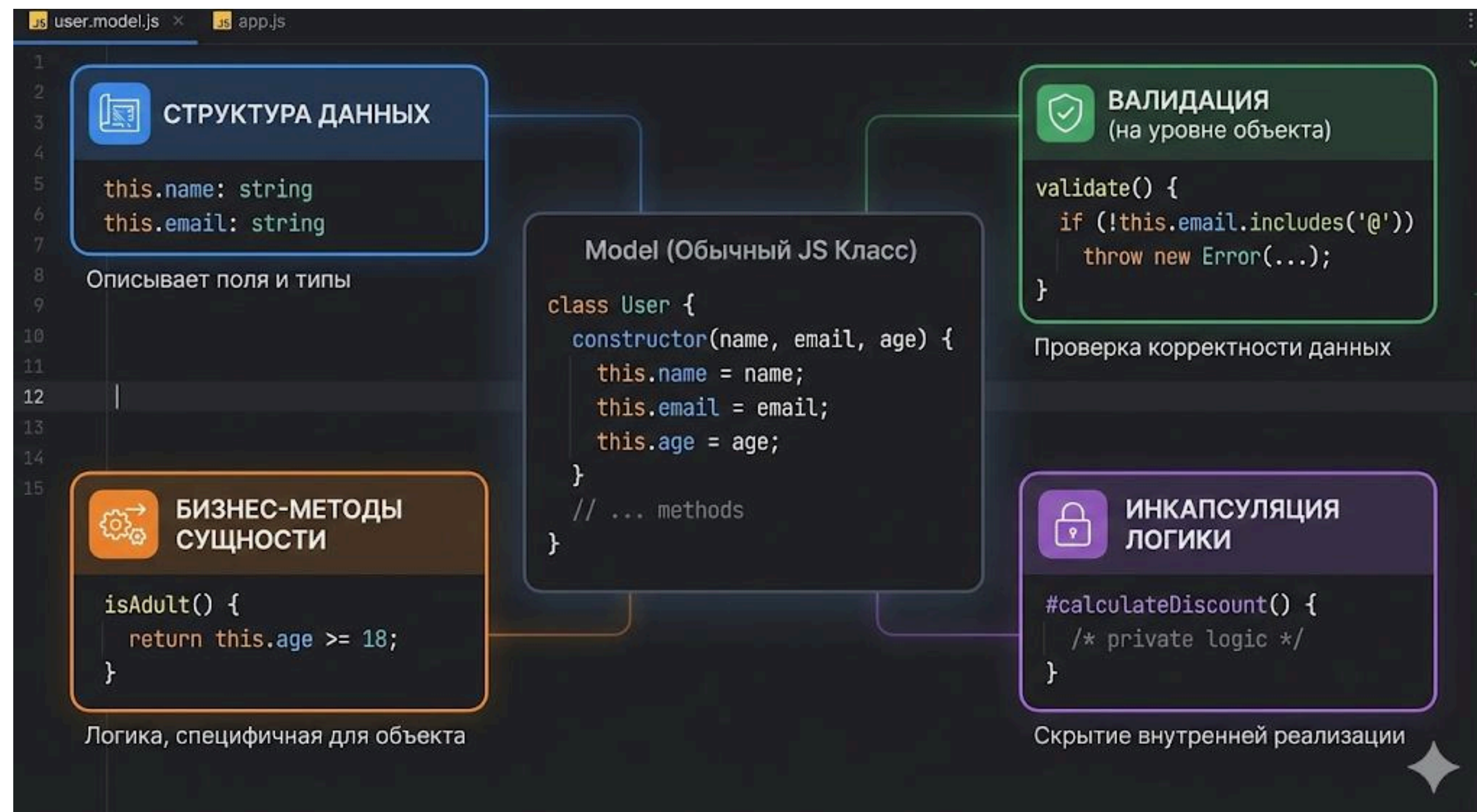
2. Client → Router → Controller → Service → Model → Database

3. Особенности:

- В REST API View чаще всего — это JSON
- В SSR View — это шаблон (EJS, Pug и др.)
- Express не навязывает MVC, структура создаётся разработчиком

MODEL

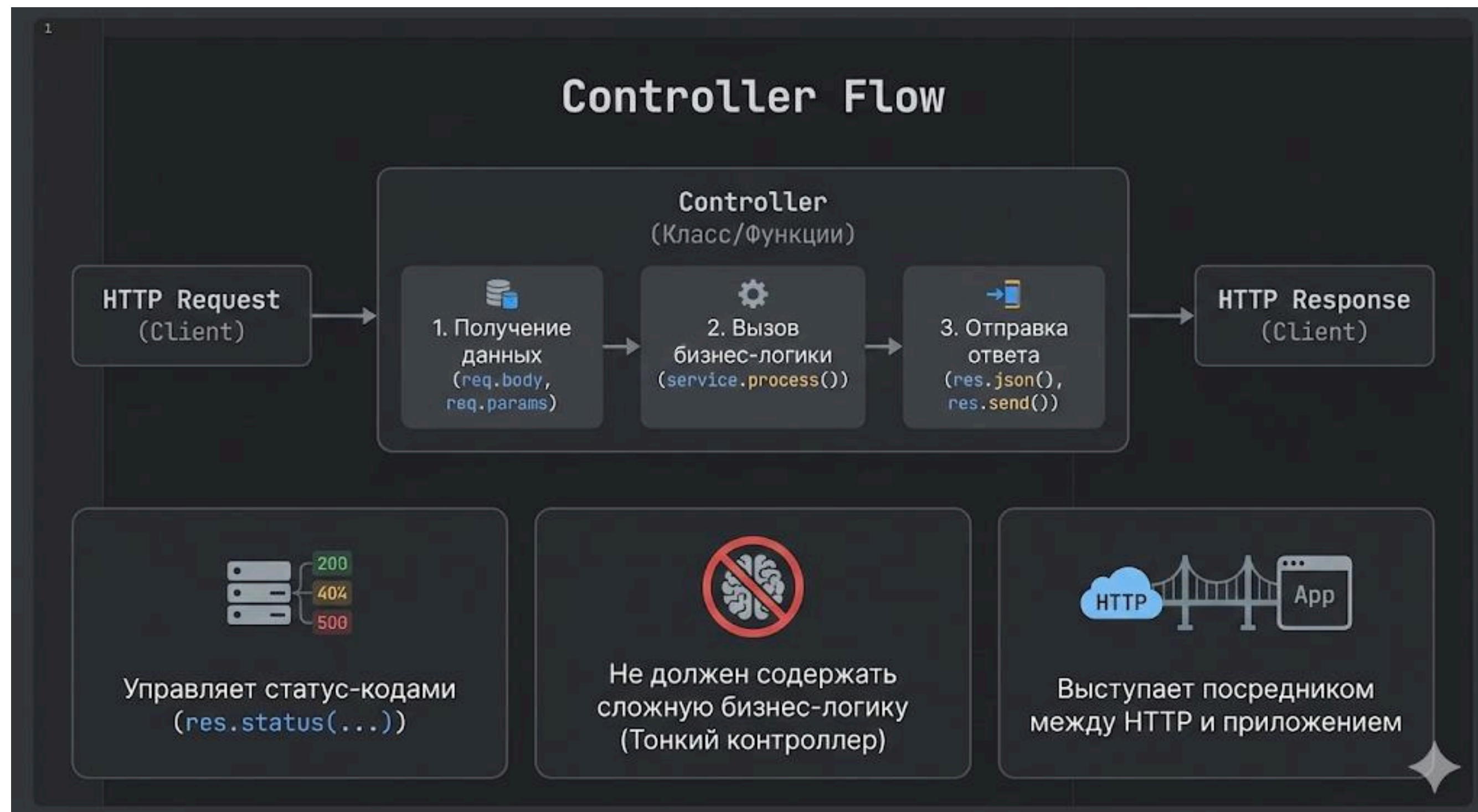
СЛОЙ РАБОТЫ С ДАННЫМИ



1. Она описывает структуру и поведение сущности на уровне бизнес-объекта.
2. Model отвечает за:
 - структуру данных
 - валидацию на уровне объекта
 - бизнес-методы сущности
 - инкапсуляцию логики

CONTROLLER

ОБРАБОТКА HTTP-ЗАПРОСА И ФОРМИРОВАНИЕ ОТВЕТА



1. Controller - это класс/набор функций который отвечает за:

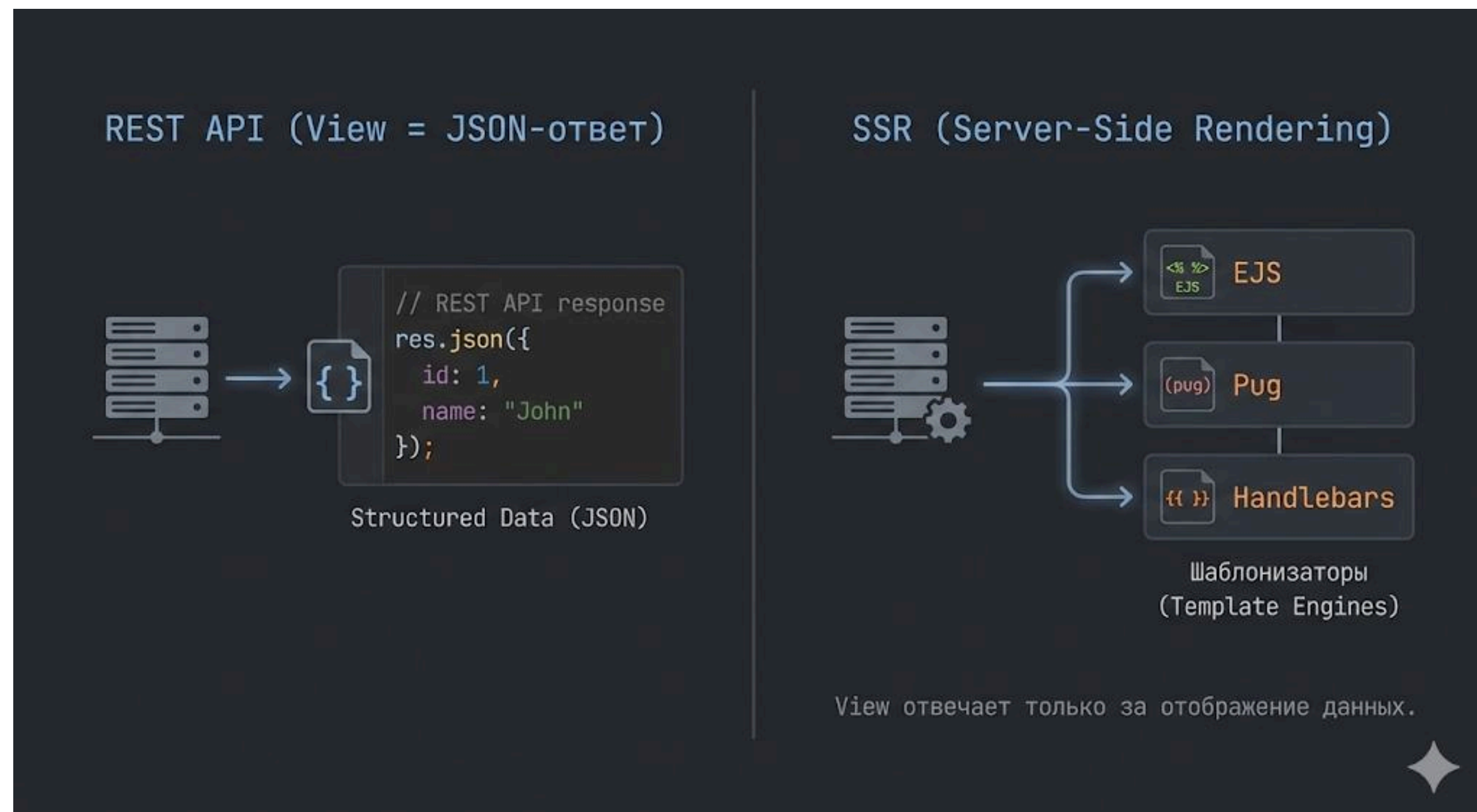
- получение данных из запроса
- вызов бизнес-логики
- отправку ответа клиенту

2. Контроллер:

- управляет статус-кодами
- не должен содержать сложную бизнес-логику
- выступает посредником между HTTP и приложением

VIEW

ПРЕДСТАВЛЕНИЕ ДАННЫХ КЛИЕНТУ



1. В Node.js возможны два варианта:

2. REST API

a. View = JSON-ответ:

`res.json({ id: 1, name: "John" });`

b. SSR (Server-Side Rendering)

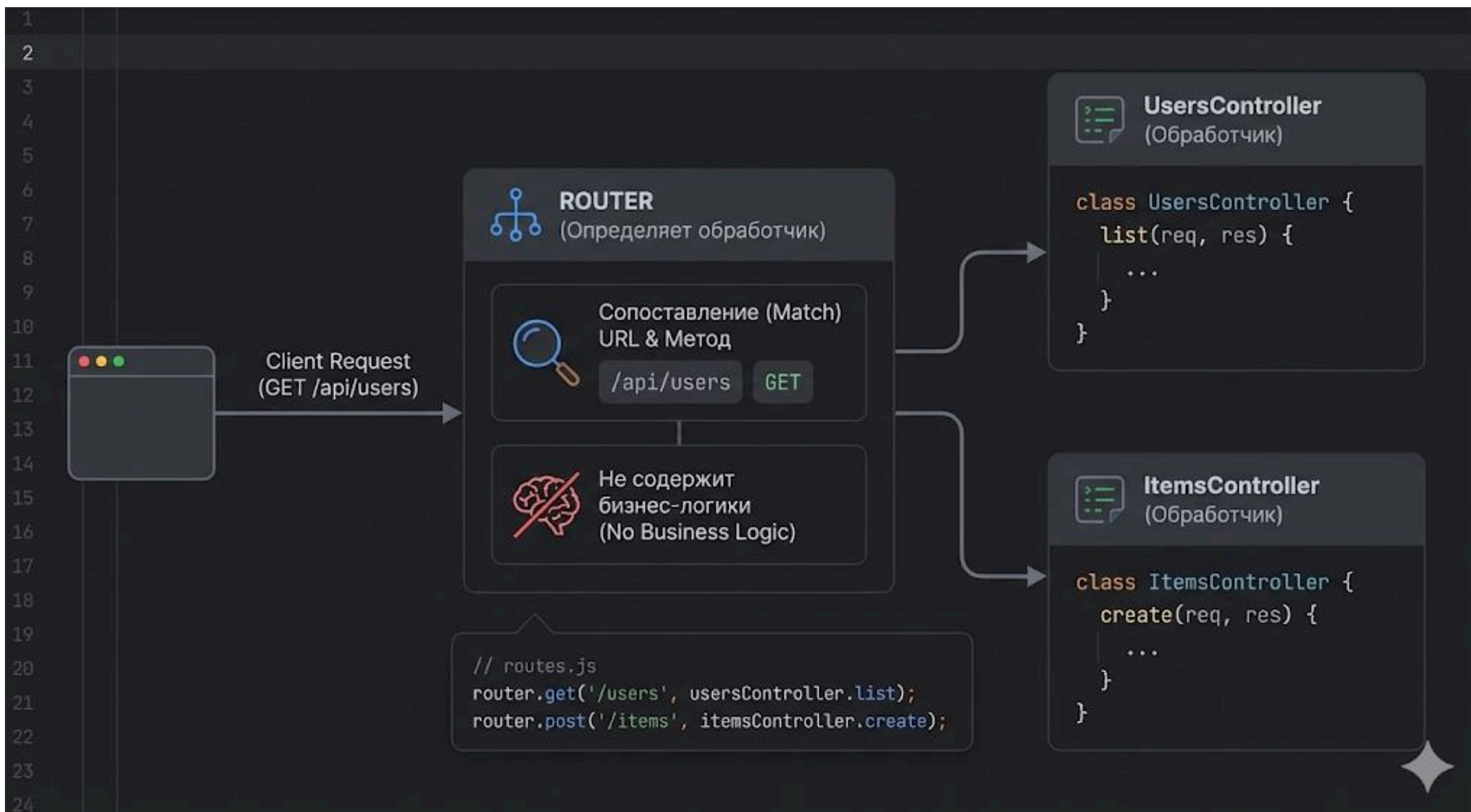
Используются шаблонизаторы:

- EJS
- Pug
- Handlebars

c. View отвечает только за отображение данных.

ROUTER

СВЯЗУЮЩЕЕ ЗВЕНО МЕЖДУ URL И КОНТРОЛЛЕРОМ



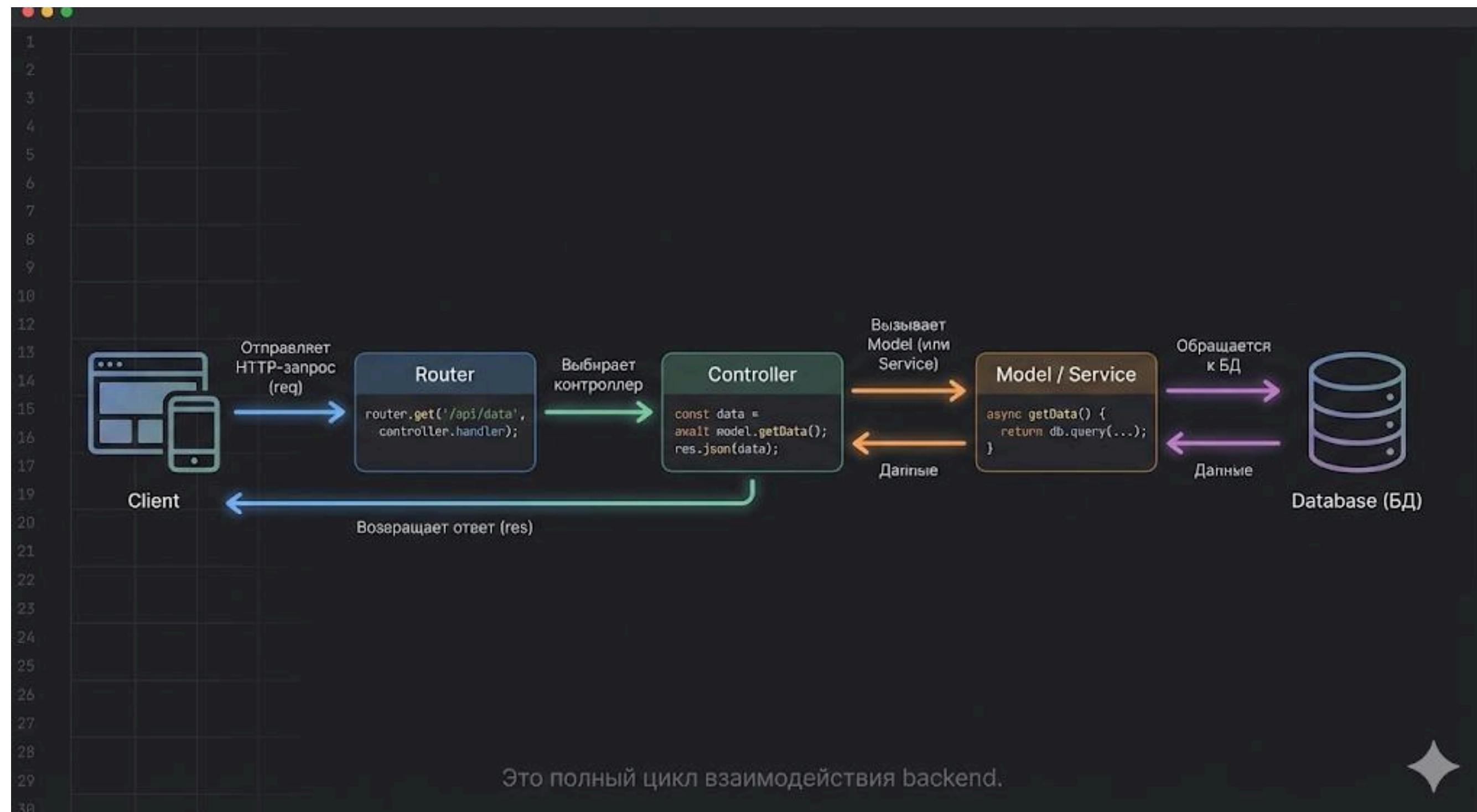
1. Router определяет, какой контроллер обрабатывает запрос.

2. Router:

- сопоставляет URL и метод
- не содержит бизнес-логики

ПОТОК ВЫПОЛНЕНИЯ ЗАПРОСА

КАК ДАННЫЕ ПРОХОДЯТ ЧЕРЕЗ MVC



1. Как данные проходят через MVC

а. Клиент отправляет HTTP-запрос

б. Router выбирает контроллер

в. Controller получает req

г. Controller вызывает Model (или Service)

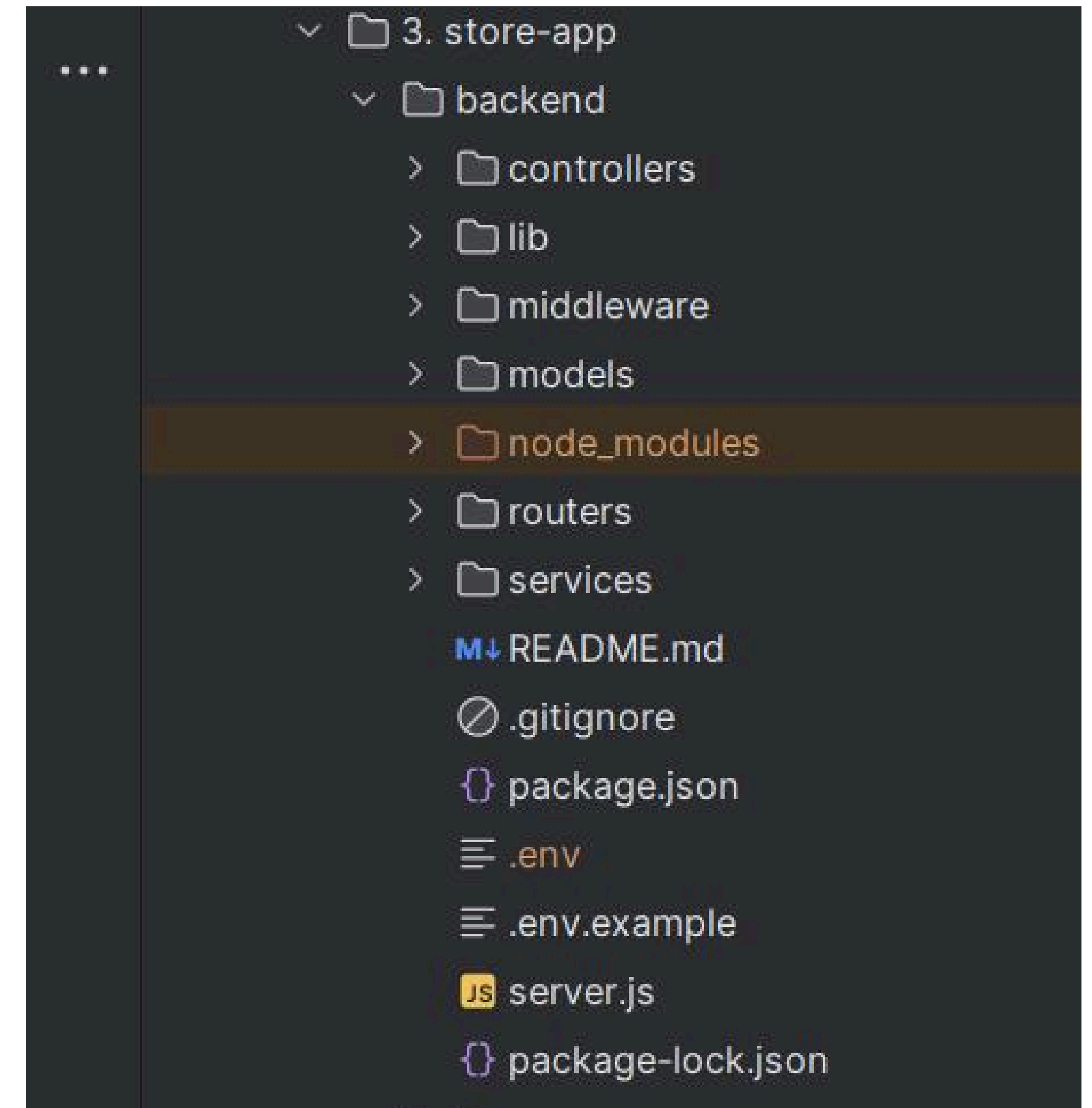
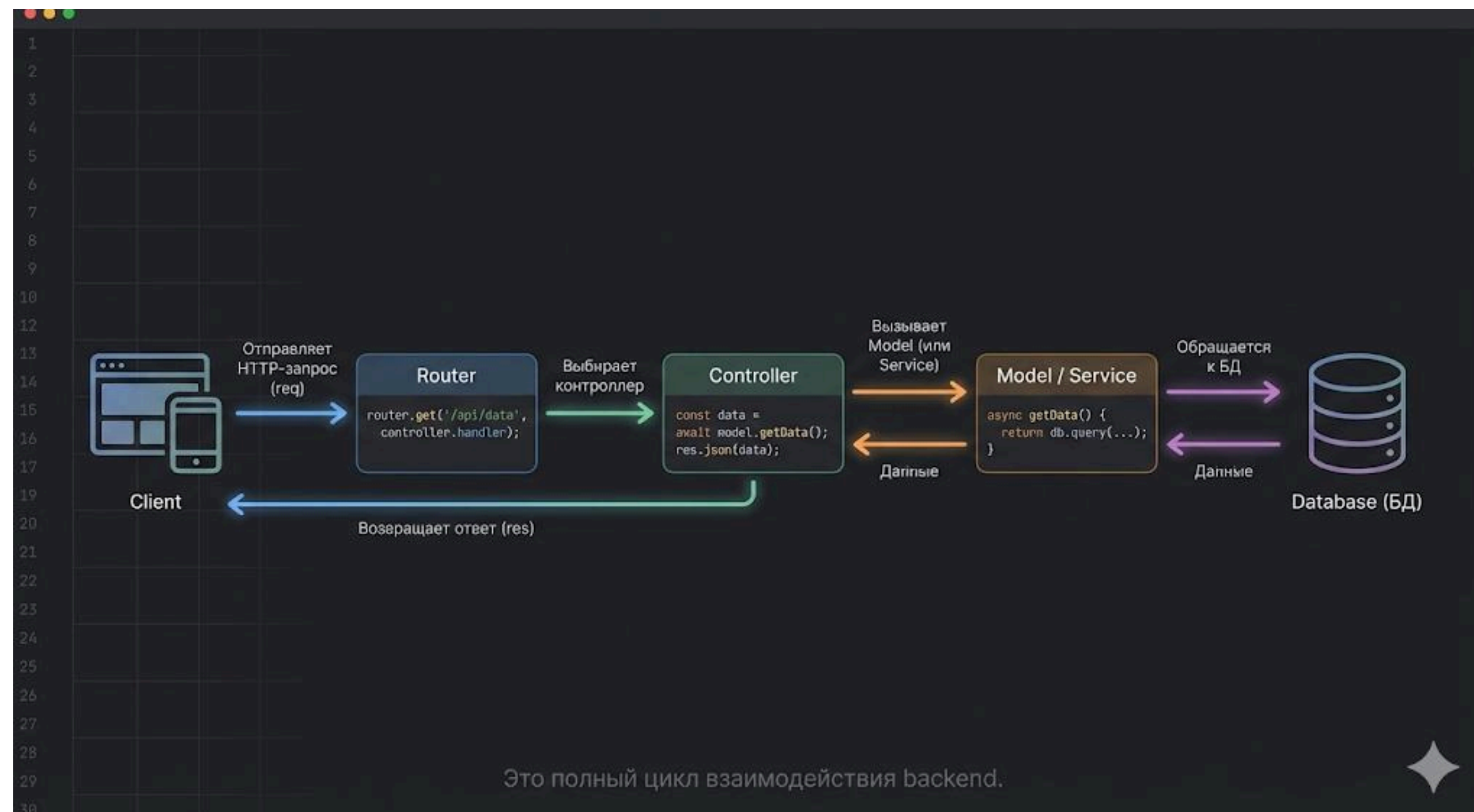
д. Model обращается к БД

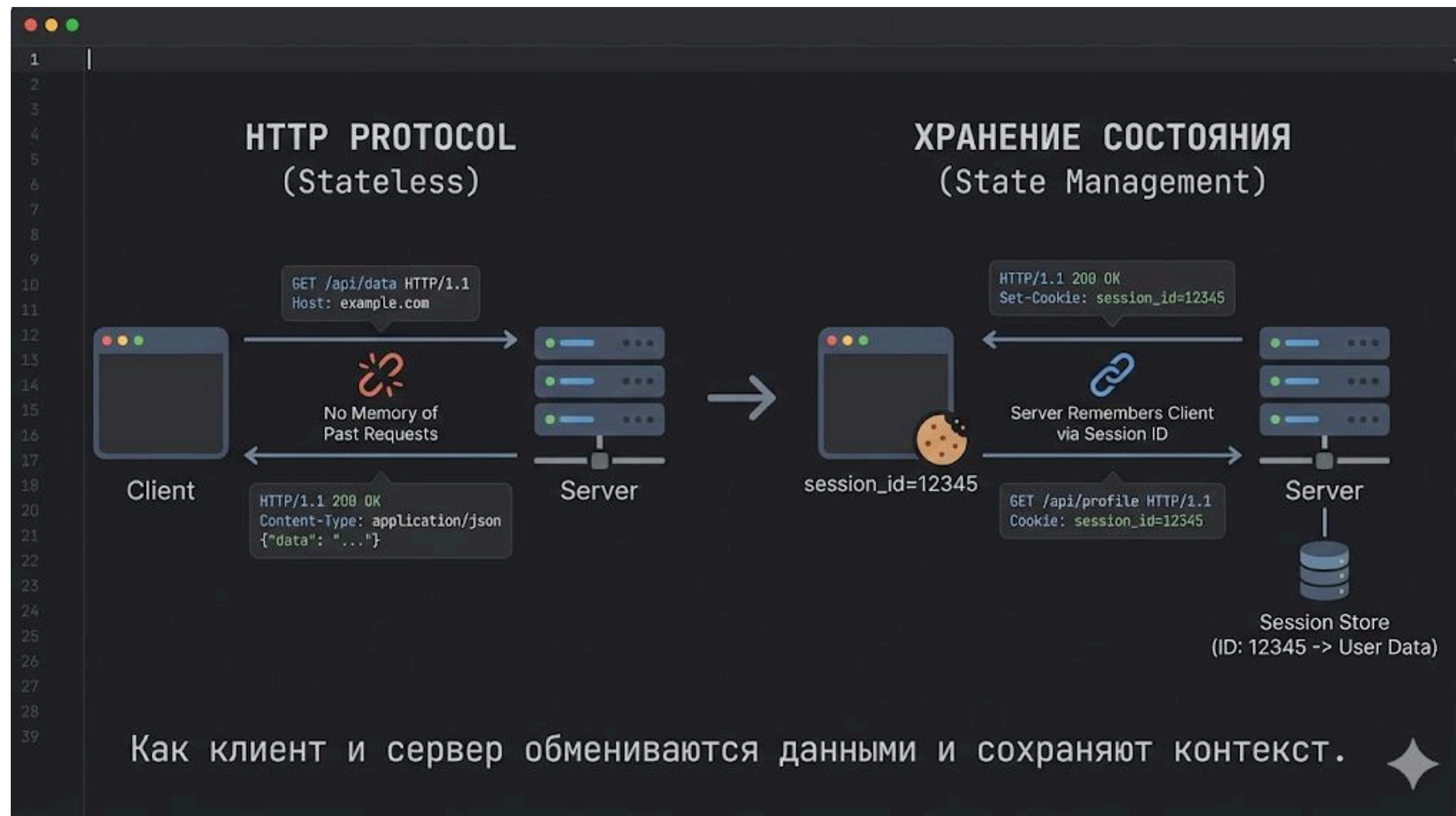
е. Controller возвращает ответ

2. Это полный цикл взаимодействия backend.

СТРУКТУРА ПРОЕКТА

ОРГАНИЗАЦИЯ КАТАЛОГОВ В MVC-ПРИЛОЖЕНИИ





МАРШРУТИЗАЦИЯ В EXPRESS.JS ROUTER И MIDDLEWARE-ФУНКЦИИ

КАК СЕРВЕР ПОНИМАЕТ, КАКОЙ КОД ВЫПОЛНЯТЬ

ЧТО ТАКОЕ МАРШРУТ В EXPRESS

КАК СЕРВЕР ПОНИМАЕТ, КАКОЙ КОД ВЫПОЛНЯТЬ



1. Маршрутизация — это механизм сопоставления:

а. HTTP-метод + URL → функция-обработчик

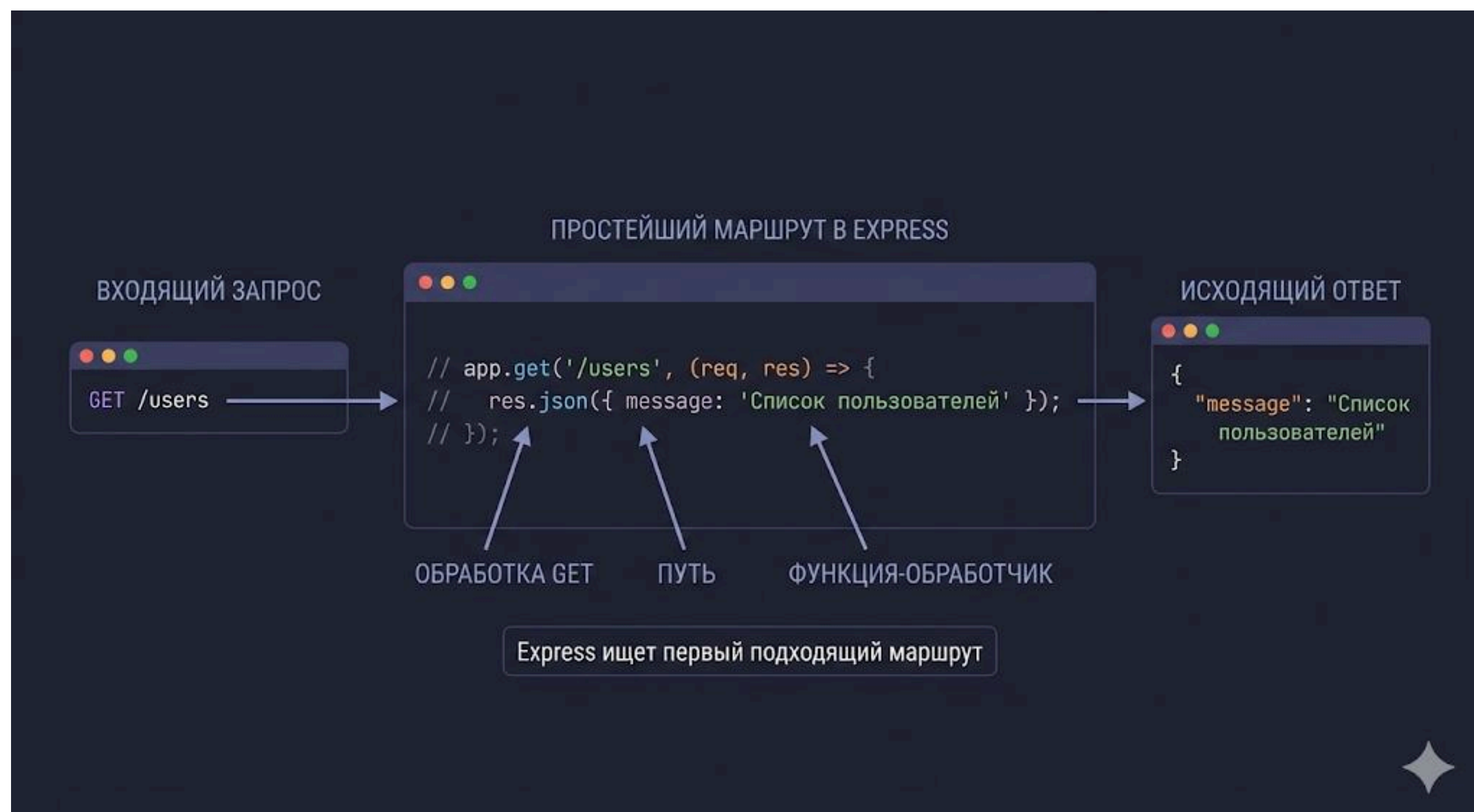
2. Пример логики:

- GET /users → получить список пользователей
- POST /users → создать пользователя
- GET /users/5 → получить пользователя с id = 5

3. Каждый маршрут — это точка входа в бизнес-логику backend.

ПРОСТЕЙШИЙ МАРШРУТ В EXPRESS

КАК СЕРВЕР ПОНИМАЕТ, КАКОЙ КОД ВЫПОЛНЯТЬ



```
app.get('/users', (req, res) => {  
  res.json({ message: 'Список пользователей' });  
});
```

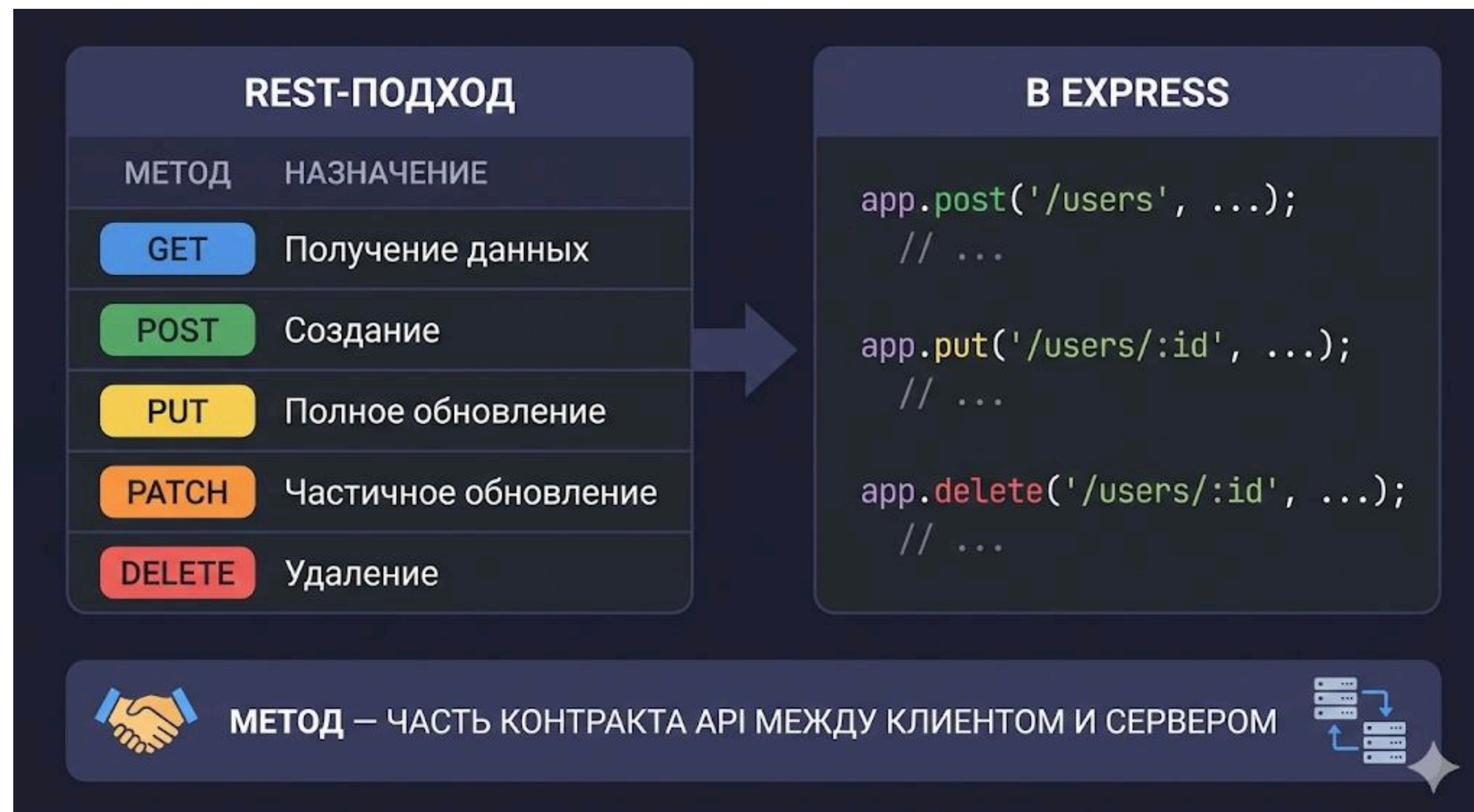
1. Здесь:

- `app.get` – обработка GET-запроса
- `/users` – путь
- функция – что делать при обращении

2. Маршрут всегда состоит из HTTP-метода (GET, POST и т.д.), URL-пути и callback-функции, которая формирует ответ.

РЕАЛИЗАЦИЯ HTTP-МЕТОДОВ

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



1. В REST-подходе каждый метод имеет смысл:

а. Метод

i. GET - Получение данных

ii. POST- Создание

iii. PUT - Полное обновление

iv. PATCH - Частичное обновление

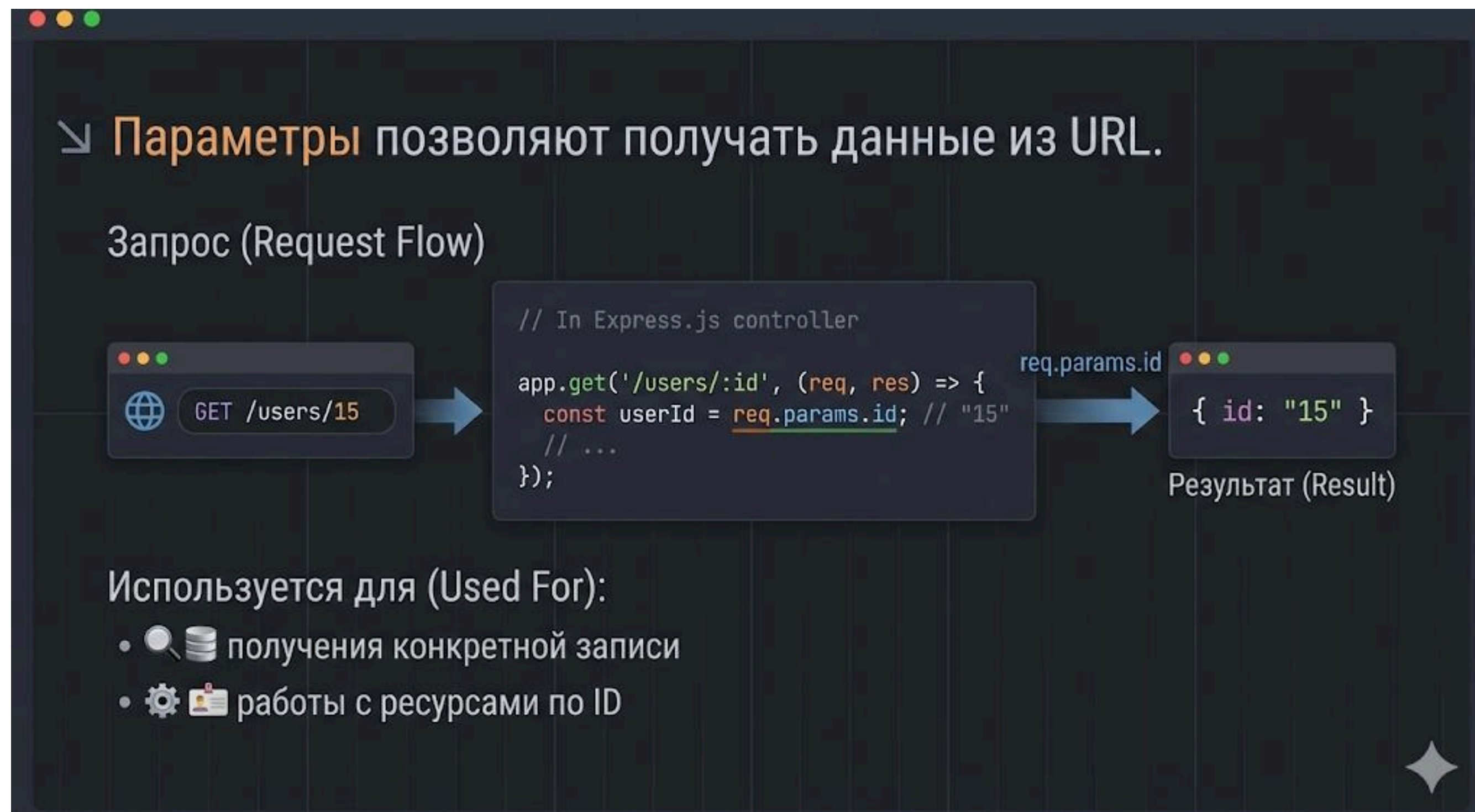
v. DELETE- Удаление

2. CRUD-операции соответствуют методам:

Create → POST, Read → GET, Update → PUT/
PATCH, Delete → DELETE.

ПАРАМЕТРЫ МАРШРУТА

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



1. Параметры позволяют получать данные из URL.

```
app.get('/users/:id', (req, res) => {
  console.log(req.params.id);
});
```

2. Запрос:

/users/15 → **req.params.id** = "15"

3. Используется для:

- получения конкретной записи
- работы с ресурсами по ID

QUERY-ПАРАМЕТРЫ

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ

ИСПОЛЬЗУЮТСЯ ДЛЯ:

- фильтрации
- сортировки
- пагинации

Это не идентификация ресурса, а параметры запроса к коллекции.

express.js route

```
app.get('/users', (req, res) => {  
  const role = req.query.role; // "admin"  
  const page = req.query.page; // "2"  
  
  // ... filter & paginate collection  
});
```

QUERY PARAMETERS (после ?)

"admin"

"2"

1. Query – это параметры после **?**:

2. **/users?role=admin&page=2**

3. В Express:

a. `req.query.role`

b. `req.query.page`

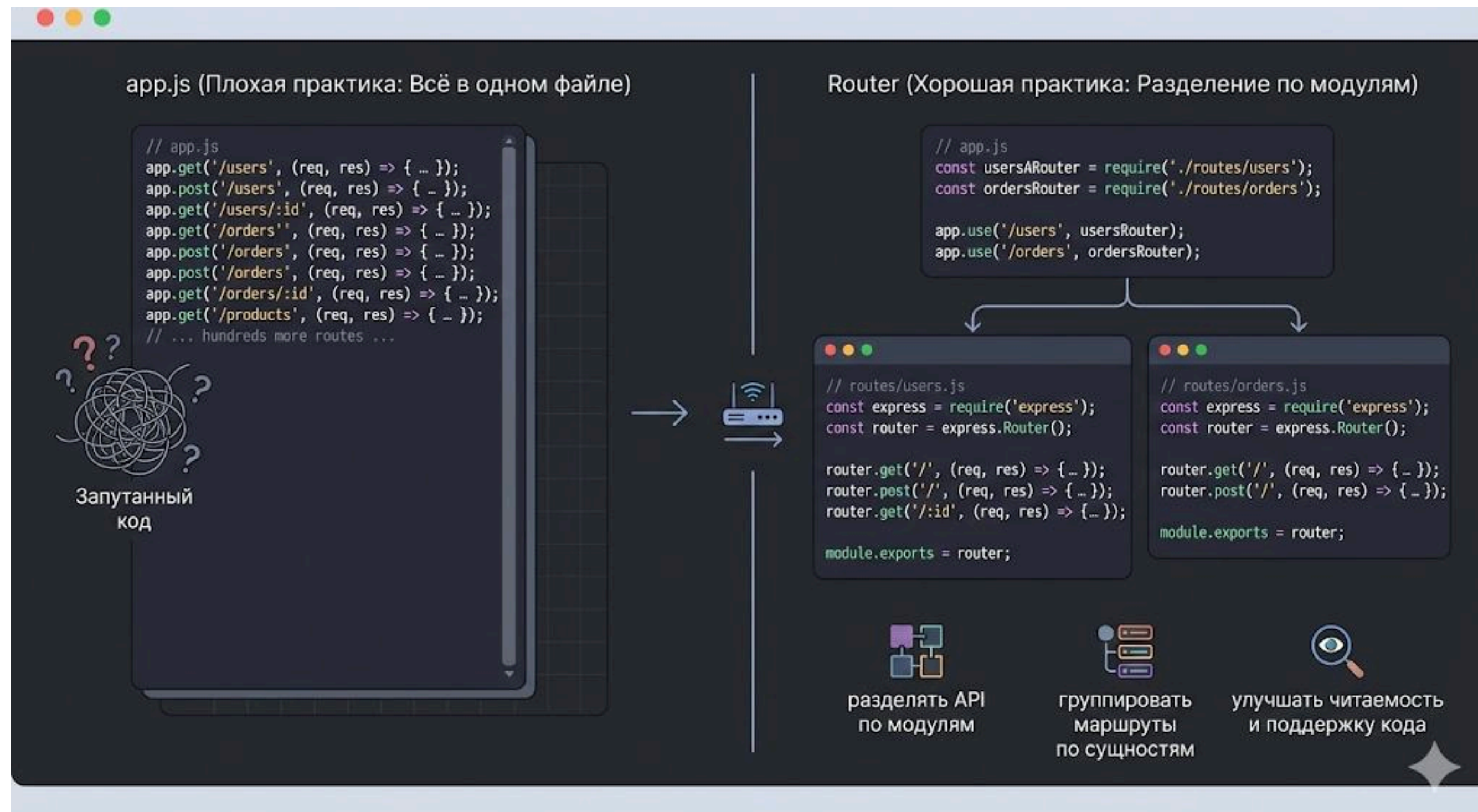
4. Используются для:

- фильтрации
- сортировки
- пагинации

5. Это не идентификация ресурса, а параметры запроса к коллекции.

ЗАЧЕМ НУЖЕН ROUTER

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



1. Когда маршрутов становится много, писать всё в `app.js` — плохая практика.

2. **Router** позволяет:

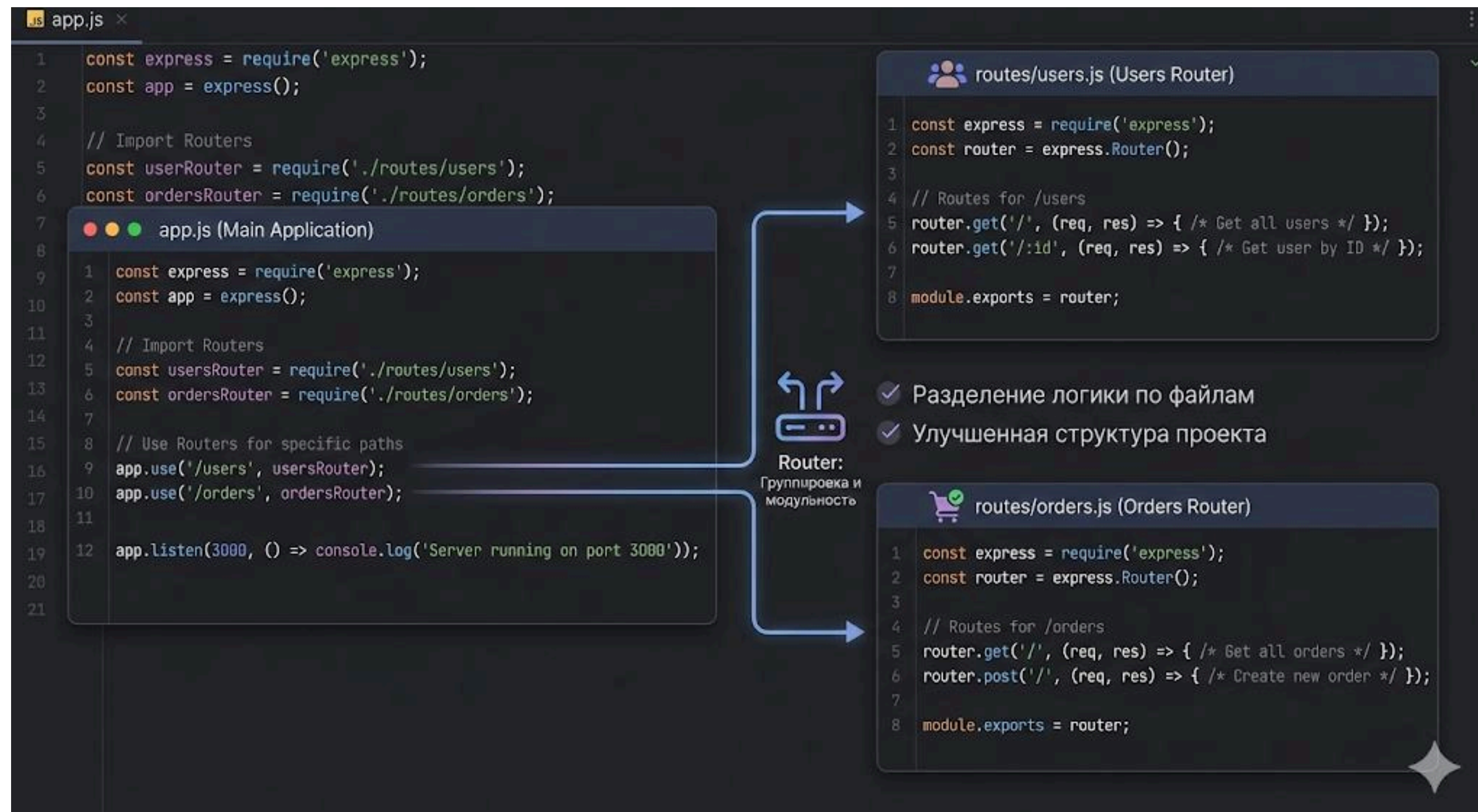
- разделять API по модулям
- группировать маршруты по сущностям
- улучшать читаемость и поддержку кода

3. Например:

- `routes/users.js`
- `routes/orders.js`

ГРУППИРОВКА МАРШРУТОВ С ROUTER

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



1. Пример:

```
const router = require('express').Router();

router.get('/', ...)
router.post('/', ...)
router.get('/:id', ...)

module.exports = router;
```

2. Подключение:

```
app.use('/users', userRouter);
```

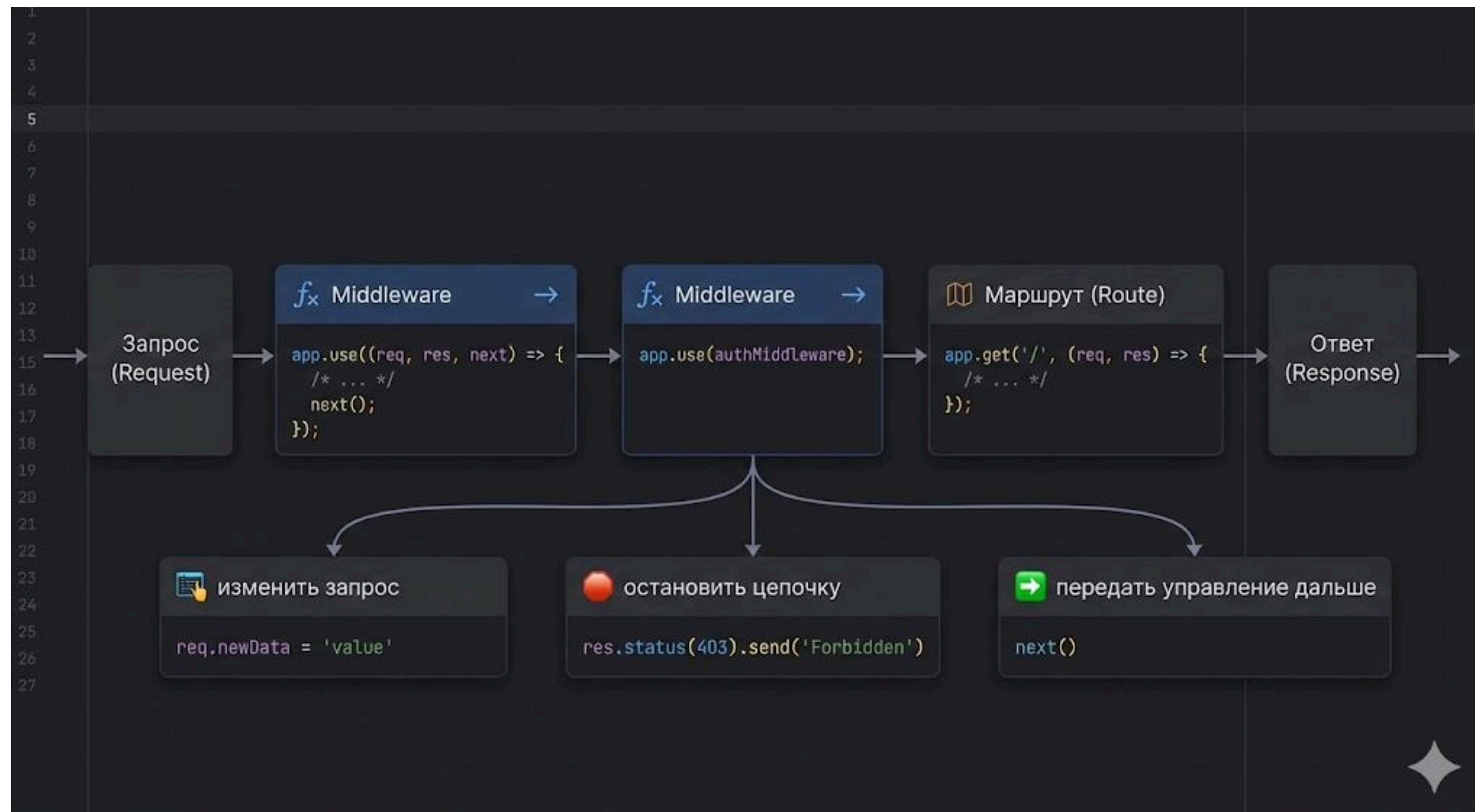
3. Теперь:

`/users` → маршруты из файла `users.js`

4. Так строится модульная архитектура backend.

ЧТО ТАКОЕ MIDDLEWARE

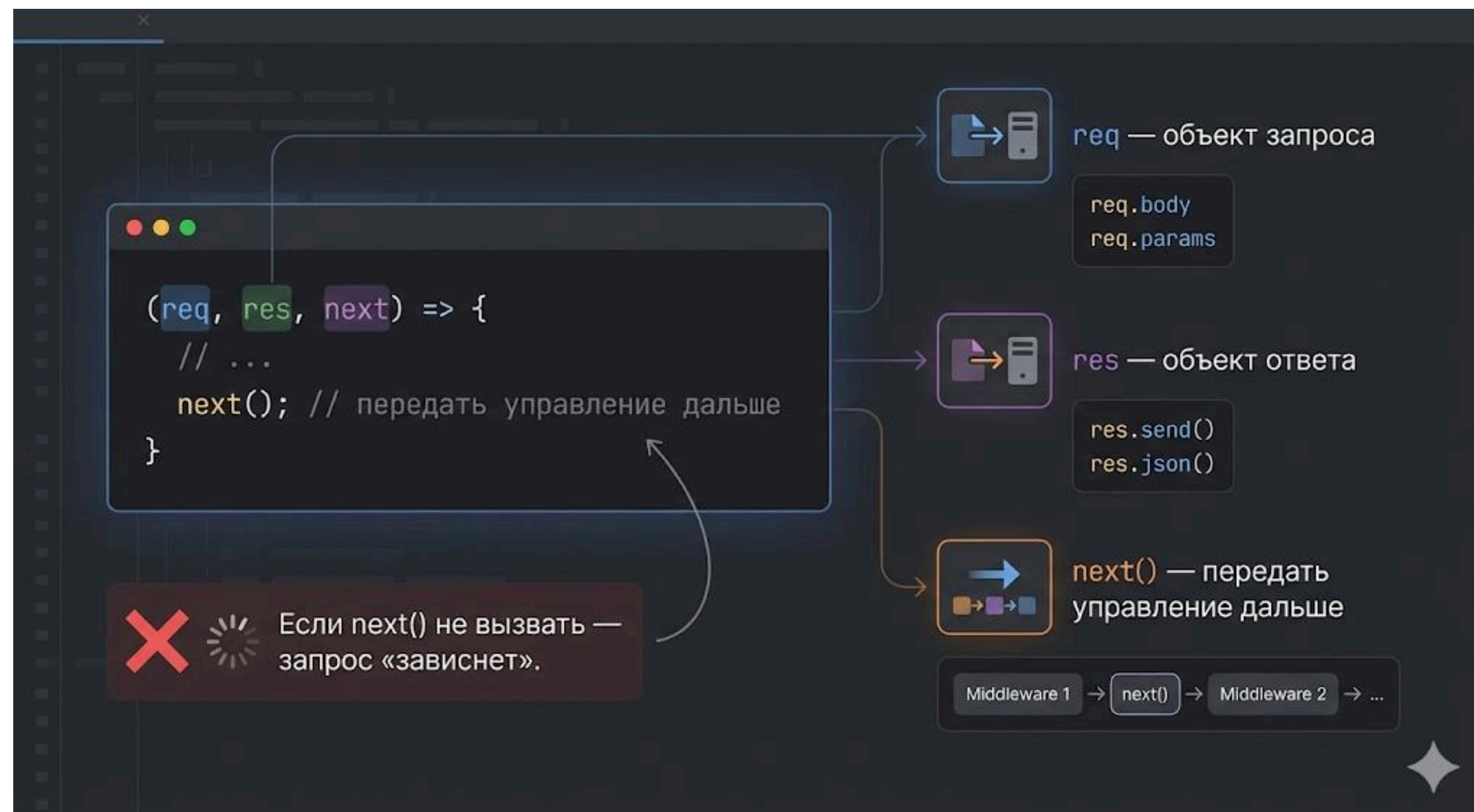
HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



1. Middleware – это функция, которая:
2. выполняется между получением запроса и отправкой ответа
3. Схема:
4. Запрос → Middleware → Middleware → Маршрут → Ответ
5. Каждый middleware может:
 - изменить запрос
 - остановить цепочку
 - передать управление дальше

СИГНАТУРА MIDDLEWARE

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



1. `(req, res, next) => { ... }`

а. `req` — объект запроса

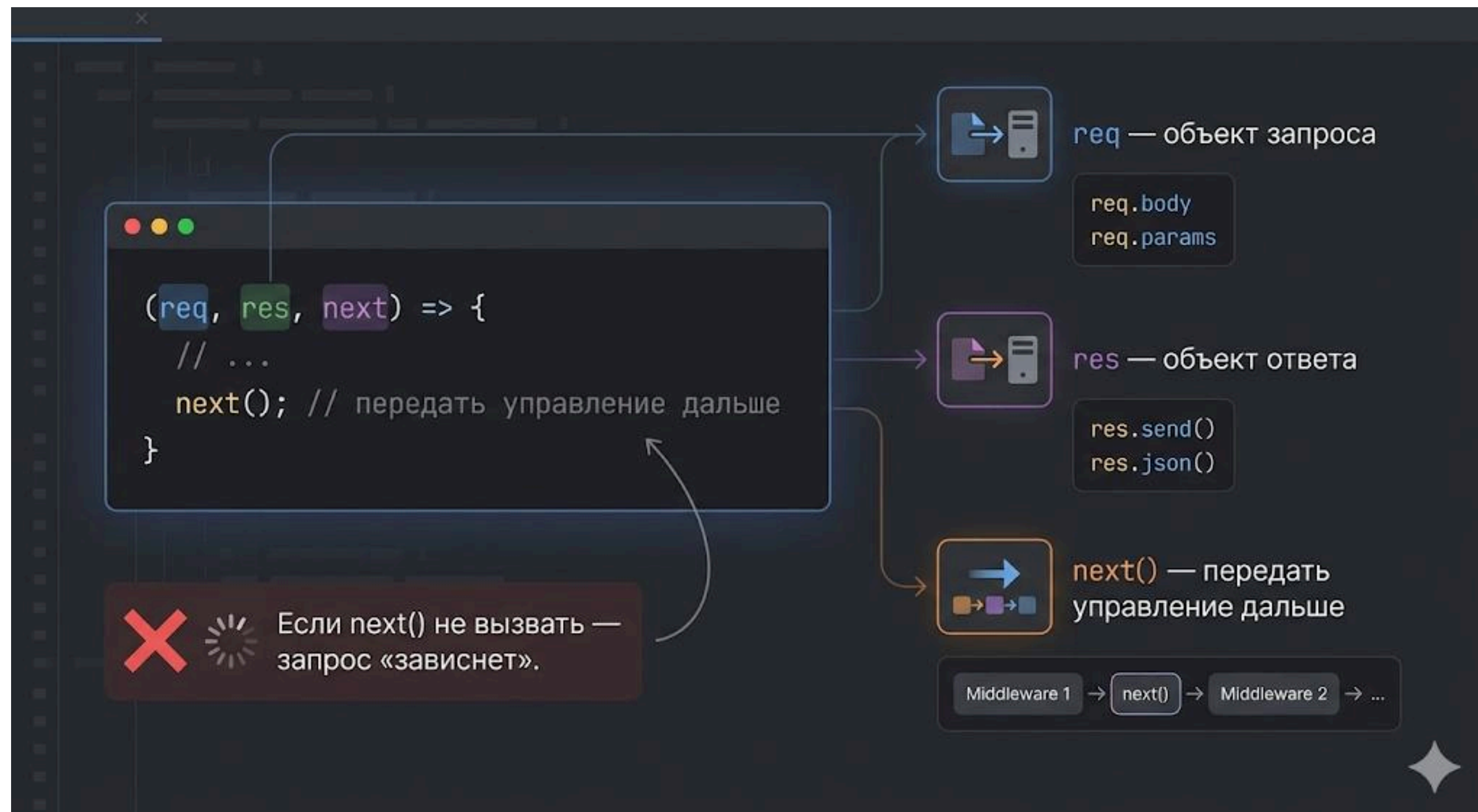
б. `res` — объект ответа

с. `next()` — передать управление дальше

2. Если `next()` не вызвать — запрос «зависнет».

ДЛЯ ЧЕГО ИСПОЛЬЗУЮТ MIDDLEWARE

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



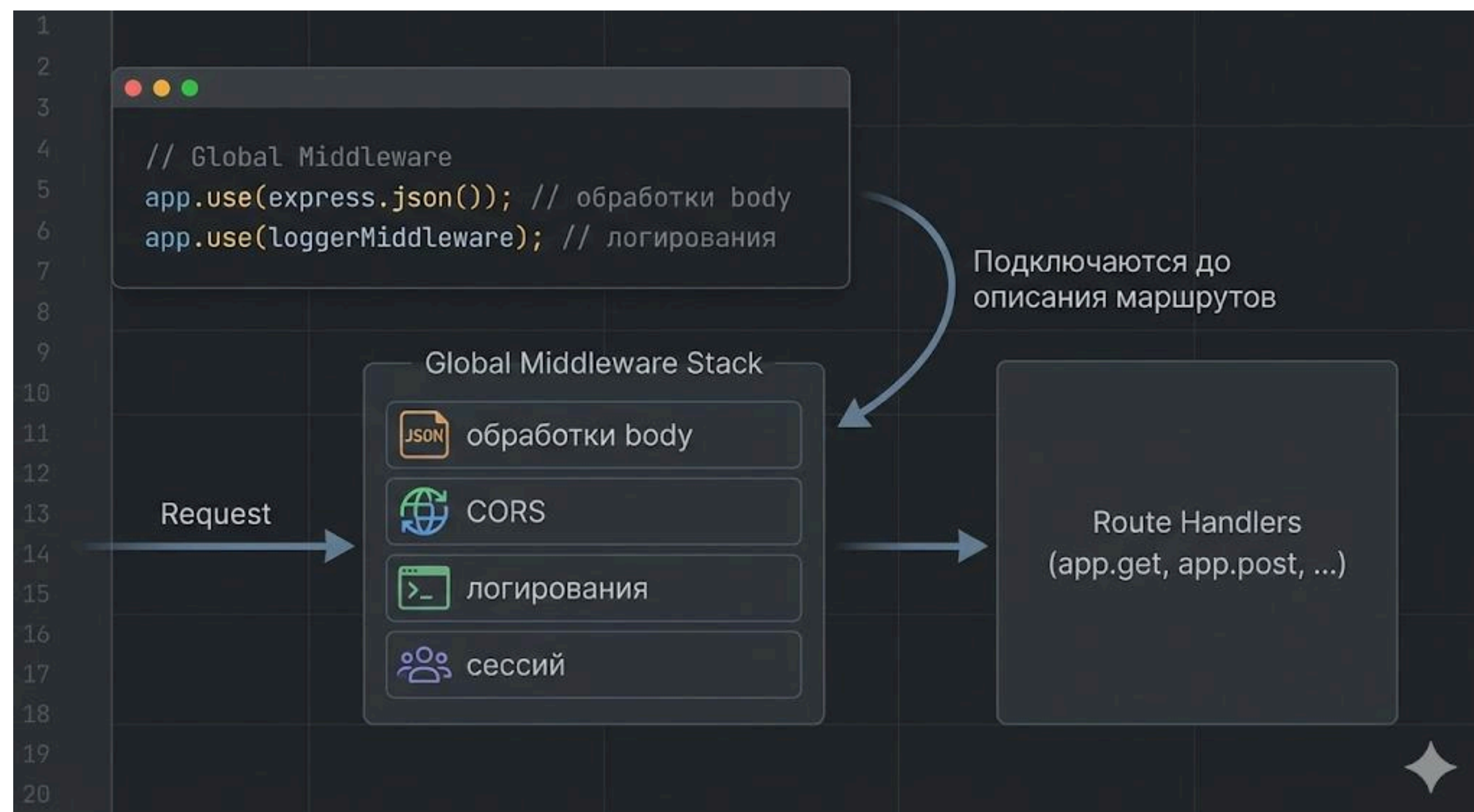
1. Типичные задачи:

- логирование запросов
- проверка авторизации
- валидация данных
- обработка ошибок
- парсинг JSON

2. Middleware — это инфраструктурная логика, а не бизнес-логика.

ГЛОБАЛЬНЫЕ MIDDLEWARE

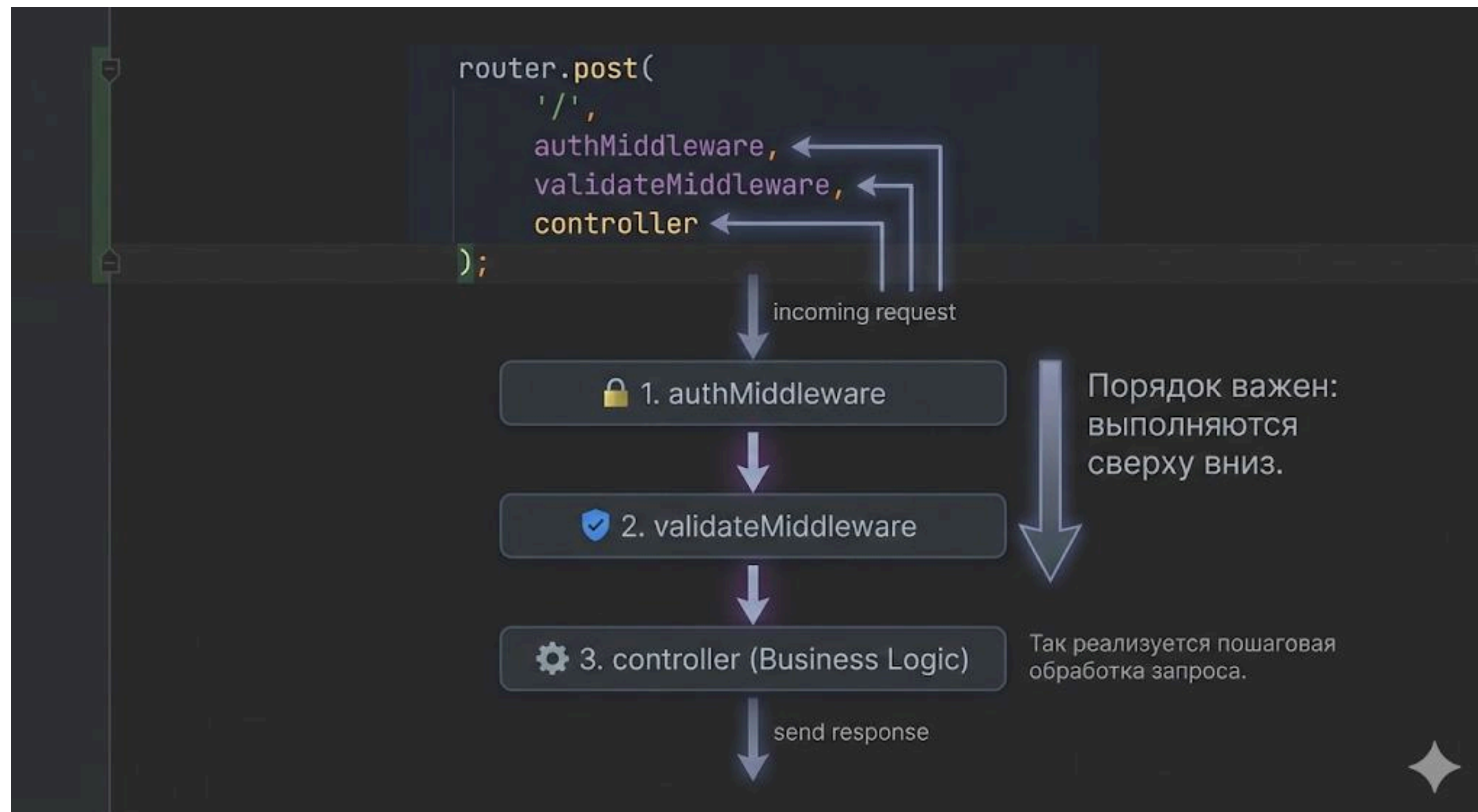
HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



1. `app.use(express.json());`
2. `app.use(loggerMiddleware);`
3. Используются для:
 - а. обработки body
 - б. CORS
 - в. логирования
 - г. сессий
4. Подключаются до описания маршрутов.
5. CORS (Cross-Origin Resource Sharing) – это механизм браузера, который блокирует запросы с одного домена к другому, если сервер явно не разрешил такой обмен.

MIDDLEWARE НА УРОВНЕ МАРШРУТА

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



1. Можно применять только к отдельным маршрутам:

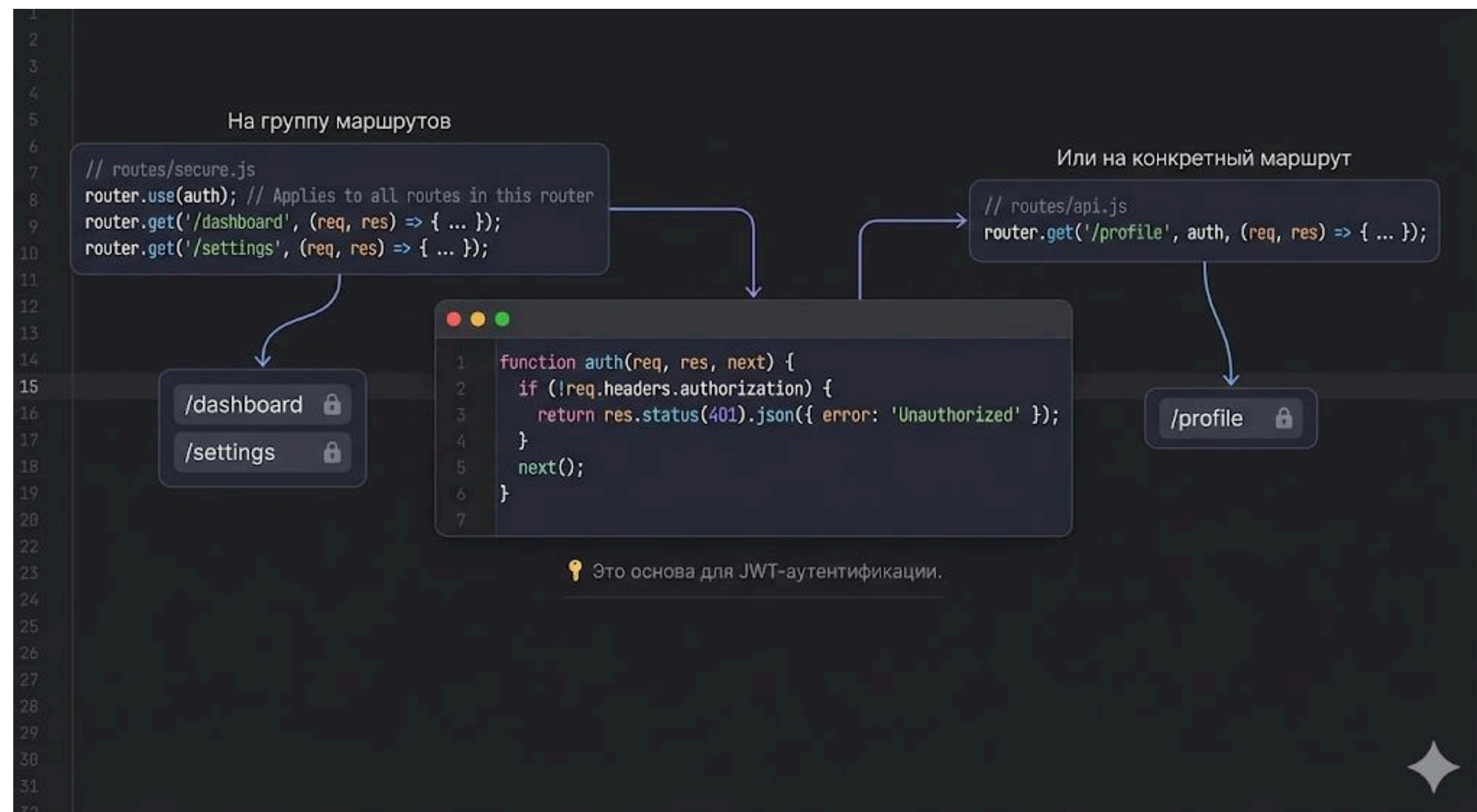
```
router.post(  
  '/',  
  authMiddleware,  
  validateMiddleware,  
  controller  
);
```

2. Порядок важен: middleware выполняются сверху вниз.

3. Так реализуется пошаговая обработка запроса.

ПРИМЕР: ПРОВЕРКА АВТОРИЗАЦИИ

HTTP-МЕТОДЫ И CRUD-ОПЕРАЦИИ



```
function auth(req, res, next) {  
  if (!req.headers.authorization) {  
    return res.status(401).json({ error: 'Unauthorized' });  
  }  
  next();  
}
```

1. Такой middleware можно ставить:
 - на группу маршрутов
 - или на конкретный маршрут
2. Это основа для JWT-аутентификации.



ВОПРОСЫ?

- ПО ТЕМАМ ДИСЦИПЛИНЫ?
- ПО ПРАКТИЧЕСКИМ/ИТОГОВЫМ РАБОТАМ?
- ПО ОРГАНИЗАЦИОННЫМ МОМЕНТАМ?